

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Task Management System**.

In this article, we'll guide you through a clear 9-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- The system should allow users to create, update, and delete tasks with title, description, due date, priority, and status.
- The system should support composite tasks where a task can contain multiple subtasks (recursive structure).
- Users should be able to assign tasks to other users and update task details including title, description, due date, and priority.
- The system should implement a state machine for task workflow with state-specific behavior.
- Users should be able to view tasks assigned to them, tasks they have created, and check delayed tasks.
- The system should support searching and filtering tasks with flexible sorting strategies (Strategy Pattern).
- The system should automatically log all task changes and notify observers of status updates (Observer Pattern).
- Users should be able to categorize tasks using tags and add comments for collaboration.

- The system should handle concurrent access to tasks and ensure data consistency.
- Users should be able to view task distribution by priority, status, and tags.

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Edge Cases to Consider

- Task Deletion with Subtasks.
 - Concurrent Task Updates.
 - Task Priority Conflicts.
 - State Transition Rules.
-

Step 2: Identify Core Entities

1. Task (Core Entity)

- id: int [PK]
- title: String
- description: String
- dueDate: LocalDateTime
- priority: Priority (Enum: LOW, MEDIUM, HIGH, URGENT)
- status: TaskStatus (Enum: TODO, IN_PROGRESS, REVIEW, COMPLETED, CANCELLED)
- assigneeId: int [FK to User]
- creatorId: int [FK to User]
- parentTaskId: int [FK to Task – for subtasks]

- tags: List<String>
- createdAt: LocalDateTime
- updatedAt: LocalDateTime

2. User

- id: int [PK]
- username: String
- email: String
- role: UserRole (Enum: USER, ADMIN)

3. TaskStatus (Enum)

- TODO
- IN_PROGRESS
- REVIEW
- COMPLETED
- CANCELLED

4. Priority (Enum)

- LOW
- MEDIUM
- HIGH
- URGENT

5. Comment

- id: int [PK]
- taskId: int [FK to Task]
- userId: int [FK to User]
- content: String
- createdAt: LocalDateTime

6. TaskChangeLog (Audit Trail)

- id: int [PK]
- taskId: int [FK to Task]

- userId: int [FK to User]
- changeType: ChangeType (Enum: CREATED, UPDATED, STATUS_CHANGED, ASSIGNED)
- oldValue: String
- newValue: String
- timestamp: LocalDateTime

7. ChangeType (Enum)

- CREATED
- UPDATED
- STATUS_CHANGED
- ASSIGNED
- PRIORITY_CHANGED

8. TaskSubscription (Observer Pattern)

- id: int [PK]
- userId: int [FK to User]
- taskId: int [FK to Task]
- isActive: boolean

10. TaskSearchCriteria

- assigneeld: int
- creatorId: int
- priority: Priority
- status: TaskStatus
- dueDateRange: DateRange
- tags: List<String>
- hasSubtasks: boolean

11. DateRange

- startDate: LocalDateTime
- endDate: LocalDateTime

Interview Tip: Clearly identifying entities and enums at this stage helps define relationships (like Task ↔ Comment, Task ↔ User) and ensures smooth implementation of Task Tracking, Subscription, and Audit mechanisms in the system.

Step 3: Visualize Interaction Flows

1. Task Creation Flow:

- **POST** /api/tasks →
 - TaskController.createTask() →
 - TaskService.createTask() →
 - Validate user permissions →
 - Create task →
 - If parent task exists, validate hierarchy →
 - Return task
-

2. Task Update Flow:

- **PUT** /api/tasks/{id} →
- TaskController.updateTask() →
- TaskService.updateTask() →
- Validate permissions →
- Update task →
- Update subtask priorities if needed →
- Return updated task

3. Task Status Change Flow:

- **PUT** */api/tasks/{id}/status* →
 - TaskStateController.updateTaskStatus() →
 - TaskStateService.updateTaskStatus() →
 - Validate state transition →
 - Update status →
 - Return success
-

4. Task Search & Sort Flow:

- **GET** */api/tasks/search* →
 - TaskController.searchTasks() →
 - TaskService.searchTasks() →
 - Apply search criteria →
 - Apply sorting strategy →
 - Return results with recursive subtask structure
-

5. Subtask Management Flow:

- **POST** */api/tasks/{id}/subtasks* →
- TaskController.addSubtask() →
- TaskService.addSubtask() →
- Validate parent task →
- Create subtask →
- Return subtask

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Service Layer → Domain Layer

Each layer has a distinct role:

- **Client/UI Layer:** Provides the interface for end users or external systems to interact with the task management system (e.g., web app, mobile app, APIs).
 - **Controller Layer:** Acts as the entry point for user requests, orchestrates calls to services, and converts input/output between the UI and business logic (e.g., creating tasks, updating status, assigning users).
 - **Service Layer:** Contains the business logic, rules, and workflows for handling task creation, state transitions, assignments, notifications, and recursive subtask management.
 - **Domain Layer:** Defines the core entities, states, and relationships that model the real-world task management system (e.g., Task, User, Comment, TaskChangeLog, TaskSubscription), independent of infrastructure or UI.
-

CONTROLLERS:

1. TaskController (Core Task Operations)

- - Task createTask(CreateTaskRequest request)
- - Task updateTask(int taskId, UpdateTaskRequest request)
- - void deleteTask(int taskId)
- - List<Task> searchTasks(TaskSearchCriteria criteria)
- - Task addSubtask(int parentTaskId, CreateTaskRequest request)

2. TaskStateController (State Management)

- - void updateTaskStatus(int taskId, TaskStatus newStatus)

3. TaskAssignmentController (Assignment Operations)

- - void assignTask(int taskId, int assigneeId)

4. TaskNotificationController (Observer Pattern)

- - void subscribeToTask(int taskId, int userId)
 - - void unsubscribeFromTask(int taskId, int userId)
 - - List<TaskChangeLog> getTaskHistory(int taskId)
-

SERVICES:

1. TaskService (Core Business Logic)

- - Task createTask(CreateTaskRequest request)
- - Task updateTask(int taskId, UpdateTaskRequest request)
- - void deleteTask(int taskId)
- - List<Task> searchTasks(TaskSearchCriteria criteria)
- - Task addSubtask(int parentTaskId, CreateTaskRequest request)

2. TaskStateService (State Management)

- - void updateTaskStatus(int taskId, TaskStatus newStatus)

- - boolean isValidTransition(TaskStatus currentStatus, TaskStatus newStatus)

3. TaskAssignmentService (Assignment Logic)

- - void assignTask(int taskId, int assigneeId)

4. TaskNotificationService (Observer Pattern)

- - void subscribeToTask(int taskId, int userId)
- - void unsubscribeFromTask(int taskId, int userId)
- - void notifySubscribers(int taskId, ChangeType changeType, String oldValue, String newValue)
- - List<TaskChangeLog> getTaskHistory(int taskId)

REPOSITORIES:

1. TaskRepository

- - Task save(Task task)
- - Task findById(int taskId)
- - List<Task> findByAssignee(int assigneeId)
- - List<Task> findByParentTask(int parentTaskId)
- - List<Task> search(TaskSearchCriteria criteria)

2. UserRepository

- - User findById(int userId)

3. CommentRepository

- - Comment save(Comment comment)
- - List<Comment> findByTaskId(int taskId)

4. TaskChangeLogRepository

- - TaskChangeLog save(TaskChangeLog log)

- - List<TaskChangeLog> findByTaskId(int taskId)

5. TaskSubscriptionRepository

- - TaskSubscription save(TaskSubscription subscription)
 - - List<TaskSubscription> findByTaskId(int taskId)
 - - List<TaskSubscription> findByUserId(int userId)
-

STATE PATTERN IMPLEMENTATION:

1. TaskState (Interface)

- + boolean canTransitionTo(TaskStatus newStatus)
- + void performTransition(Task task, TaskStatus newStatus)
- + String getStateName()

2. TodoState (Concrete State)

- + boolean canTransitionTo(TaskStatus newStatus) *// Can go to IN_PROGRESS, CANCELLED*
- + void performTransition(Task task, TaskStatus newStatus) *// Update status*
- + String getStateName() *// Returns "TODO"*

3. InProgressState (Concrete State)

- + boolean canTransitionTo(TaskStatus newStatus) *// Can go to REVIEW, CANCELLED*
- + void performTransition(Task task, TaskStatus newStatus) *// Update status*
- + String getStateName() *// Returns "IN_PROGRESS"*

4. ReviewState (Concrete State)

- + boolean canTransitionTo(TaskStatus newStatus) *// Can go to COMPLETED, IN_PROGRESS*

- + void performTransition(Task task, TaskStatus newStatus) // *Update status*
- + String getStateName() // *Returns "REVIEW"*

5. CompletedState (Concrete State)

- + boolean canTransitionTo(TaskStatus newStatus) // *Can go to IN_PROGRESS (reopen)*
- + void performTransition(Task task, TaskStatus newStatus) // *Update status*
- + String getStateName() // *Returns "COMPLETED"*

6. CancelledState (Concrete State)

- + boolean canTransitionTo(TaskStatus newStatus) // *Can go to TODO (reactivate)*
- + void performTransition(Task task, TaskStatus newStatus) // *Update status*
- + String getStateName() // *Returns "CANCELLED"*

STRATEGY PATTERN IMPLEMENTATION (Sorting):

1. TaskSortingStrategy (Interface)

- + List<Task> sort(List<Task> tasks)
- + String getStrategyName()

2. PrioritySortingStrategy (Concrete Strategy)

- + List<Task> sort(List<Task> tasks) // *Sort by priority (URGENT → HIGH → MEDIUM → LOW)*
- + String getStrategyName() // *Returns "PRIORITY"*

3. DueDateSortingStrategy (Concrete Strategy)

- + List<Task> sort(List<Task> tasks) // *Sort by due date (earliest first)*

- + String getStrategyName() // Returns "DUE_DATE"

4. CreatedDateSortingStrategy (Concrete Strategy)

- + List<Task> sort(List<Task> tasks) // Sort by creation date (newest first)
- + String getStrategyName() // Returns "CREATED_DATE"

5. TaskSortingContext (Context)

- - TaskSortingStrategy strategy
- + void setSortingStrategy(TaskSortingStrategy strategy)
- + List<Task> sortTasks(List<Task> tasks)

6. TaskSearchCriteria (Enhanced with Sorting)

- - sortBy: String ("priority", "dueDate", "createdDate")
- - sortOrder: String ("asc", "desc")
- + Builder pattern methods for sorting configuration

7. TaskService Integration

- + searchTasks() method automatically applies appropriate sorting strategy
- + Runtime strategy selection based on search criteria
- + Support for ascending/descending order

OBSERVER PATTERN IMPLEMENTATION:

1. TaskSubject (Subject Interface)

- + void attach(TaskSubscriber subscriber)
- + void detach(TaskSubscriber subscriber)
- + void notifySubscribers(ChangeType changeType, String oldValue, String newValue)

2. Task (Enhanced with Observer Pattern)

- - subscribers: List<TaskSubscriber>
- + void attach(TaskSubscriber subscriber)
- + void detach(TaskSubscriber subscriber)
- + void notifySubscribers(ChangeType changeType, String oldValue, String newValue)
- + Implements TaskSubject interface

3. TaskSubscriber (Observer Interface)

- + void update(int taskId, ChangeType changeType, String oldValue, String newValue)

4. EmailSubscriber (Concrete Observer)

- - emailService: EmailService
- + void update(int taskId, ChangeType changeType, String oldValue, String newValue)

5. MobileAppSubscriber (Concrete Observer)

- - pushNotificationService: PushNotificationService
- + void update(int taskId, ChangeType changeType, String oldValue, String newValue)

RECURSIVE SUBTASK STRUCTURE:

1. Task (Enhanced with Recursive Logic)

- - id: int
- - title: String
- - description: String
- - dueDate: LocalDateTime
- - priority: Priority
- - status: TaskStatus
- - assigneeId: int

- - creatorId: int
 - - parentId: int
 - - tags: List<String>
 - - createdAt: LocalDateTime
 - - updatedAt: LocalDateTime
 - + List<Task> getSubtasks() *// Recursive method*
 - + List<Task> getAllSubtasks() *// Get all nested subtasks*
 - + boolean hasSubtasks()
 - + int getSubtaskCount()
 - + void updateSubtaskPriorities() *// Recursive priority update*
-

Step 5: Core Use Cases & Methods

1. Task Creation Use Case:

- POST /api/tasks →
 - TaskController.createTask() →
 - TaskService.createTask() →
 - Validate user permissions →
 - Create task →
 - If parent task exists, validate hierarchy →
 - Notify subscribers →
 - Return task
-

2. Task Status Update Use Case:

- PUT /api/tasks/{id}/status →
- TaskStateController.updateTaskStatus() →
- TaskStateService.updateTaskStatus() →
- Validate state transition →

- Update status →
 - Notify subscribers →
 - Return success
-

3. Task Search & Sort Use Case:

- GET /api/tasks/search →
 - TaskController.searchTasks() →
 - TaskService.searchTasks() →
 - Apply search criteria →
 - Apply sorting strategy →
 - Return results with recursive subtask structure
-

4. Subtask Management Use Case:

- POST /api/tasks/{id}/subtasks →
 - TaskController.addSubtask() →
 - TaskService.addSubtask() →
 - Validate parent task →
 - Create subtask →
 - Notify subscribers →
 - Return subtask
-

5. Task Assignment Use Case:

- PUT /api/tasks/{id}/assign →
- TaskAssignmentController.assignTask() →

- TaskAssignmentService.assignTask() →
 - Validate assignee →
 - Update assignment →
 - Notify subscribers →
 - Return success
-

6. Task Subscription Use Case:

- POST /api/tasks/{id}/subscribe →
 - TaskNotificationController.subscribeToTask() →
 - TaskNotificationService.subscribeToTask() →
 - Create subscription →
 - Return success
-

Step 6: Apply OOP Principles & Design Patterns

The system design leverages key design patterns and OOP principles to ensure robust state handling, modular structure, and easy scalability across intersections.

Design Patterns Used:

- **Repository Pattern** – for data access abstraction
- **Service Layer Pattern** – for business logic separation
- **State Pattern** – for task status management
- **Strategy Pattern** – for flexible sorting algorithms
- **Observer Pattern** – for task change notifications
- **Composite Pattern** – for task/subtask hierarchy
- **RESTful API Design** – for clean HTTP endpoints

OOP Principles Applied:

- **Single Responsibility** – each service has one clear purpose
 - **Open/Closed** – easy to extend with new sorting strategies and states (DEMONSTRATED)
 - **Encapsulation** – domain objects encapsulate their data and behavior
 - **Dependency Inversion** – services depend on abstractions, not concrete implementations
 - **Polymorphism** – different states and sorting strategies can be swapped at runtime (DEMONSTRATED)
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Task Deletion with Subtasks** – Cascade delete all subtasks recursively
 - **Concurrent Task Updates** – Use optimistic locking with version field
 - **Task Priority Conflicts** – Auto-adjust parent task priority upward if child has higher priority
 - **State Transition Rules** – Strict state machine with predefined valid transitions
-

Implementation Strategies:

- **Recursive Subtasks:** Use parentTaskId foreign key with recursive queries
- **State Management:** State pattern ensures valid transitions and state-specific behavior

- **Sorting Strategy:** Strategy pattern allows easy addition of new sorting algorithms (FULLY IMPLEMENTED)
 - **Concurrent Access:** Optimistic locking prevents data corruption
 - **Priority Management:** Recursive priority updates maintain hierarchy consistency
-

Step-8.5: Enhanced Sorting Capabilities (Implemented)

Sorting Strategies Fully Implemented:

1. Priority Sorting (Default)

- HIGH → MEDIUM → LOW priority order
- Automatic strategy selection
- Configurable ascending/descending order

2. Due Date Sorting

- Earliest due date first (ascending)
- Latest due date first (descending)
- Null-safe date handling

3. Created Date Sorting

- Newest tasks first (descending)
- Oldest tasks first (ascending)
- Automatic timestamp handling

Implementation Details:

- TaskSearchCriteria enhanced with sortBy and sortOrder fields
- Builder pattern for fluent configuration
- Automatic strategy selection in `TaskService.searchTasks()`
- Runtime strategy swapping demonstrated in simulation
- Support for both ascending and descending order

Usage Example:

- `// Sort by priority (HIGH → LOW)`
 - `TaskSearchCriteria criteria = new TaskSearchCriteria()`
 - `.sortBy("priority")`
 - `.sortOrder("desc");`
 -
 - `// Sort by due date (earliest first)`
 - `TaskSearchCriteria criteria = new TaskSearchCriteria()`
 - `.sortBy("dueDate")`
 - `.sortOrder("asc");`
 -
 - `// Sort by creation date (newest first)`
 - `TaskSearchCriteria criteria = new TaskSearchCriteria()`
 - `.sortBy("createdDate")`
 - `.sortOrder("desc");`
-

Step 9: Future Requirements

1. Advanced Task Dependencies

- Task blocking and dependencies
- Critical path analysis
- Resource allocation

2. Enhanced Collaboration

- Real-time notifications
- Task discussions and threads
- File attachments

3. Advanced Analytics

- Time tracking and estimation
- Team velocity metrics
- Project burndown charts

4. Integration Capabilities

- Calendar integration
 - Email notifications
 - Third-party tool integration
-

Summary

This simplified design focuses on 5 core API flows perfect for a 60-minute interview:

- **Task Creation:** POST /api/tasks (core task management)
 - **Task Status Update:** PUT /api/tasks/{id}/status (state management)
 - **Task Search & Sort:** GET /api/tasks/search (strategy pattern)
 - **Subtask Management:** POST /api/tasks/{id}/subtasks (recursive logic)
 - **Task Assignment:** PUT /api/tasks/{id}/assign (user management)
-

Key Features Implemented:

- Recursive task/subtask structure with Composite pattern
- State pattern for task workflow management (FULLY IMPLEMENTED)
- Strategy pattern for flexible sorting algorithms (FULLY IMPLEMENTED)
- Observer pattern for task change notifications (FULLY IMPLEMENTED)
- Clean architecture with proper separation of concerns
- RESTful API design with clear endpoints
- Comprehensive edge case handling
- Enhanced TaskSearchCriteria with sorting capabilities
- Runtime strategy swapping demonstrated in simulation

Design Pattern Highlights:

- **State Pattern** – Manages task status transitions with validation (FULLY IMPLEMENTED)
- **Strategy Pattern** – Allows easy swapping of sorting algorithms (FULLY IMPLEMENTED)
- **Observer Pattern** – Notifies interested parties of task changes (FULLY IMPLEMENTED)
- **Composite Pattern** – Handles recursive task hierarchy naturally
- **Repository Pattern** – Abstracts data access for testability
- **Builder Pattern** – TaskSearchCriteria uses fluent interface for easy configuration